

SESSION - 12

Core Components



Table of Contents

Table of Contents **1**
Core Components: An Overview ***2***
Components ***2***
Core Components ***3***
Core Components Library ***3***
Features of Core Components ***4***
Proxy Components ***4***
Client Libraries for Core Components ***7***
Exercise 2: Add core component client libraries **9**
Content Policies ***10***
Exercise 3: Add proxy components to the template **10**
Exercise 4: Add content policies to proxy components **11**
Developing Core Components ***12***
Author with Core Components ***13***
Exercise 5: Add core components **14**
Task 1: Create a page 14
Task 2: Add Core components 14

Core Components: An Overview

Introduction

Adobe Experience Manager (AEM) provides a wide range of component implementations on a standard instance by default. Before working with components in AEM, you must understand the key concepts of a component and its types.

Objectives

After completing this module, you will be able to:

- Describe components
- Describe Core components
- Create proxy components
- Add Core component client libraries
- Describe the content policy
- Add proxy components to the template
- Add content policies to proxy components
- Author Core components

Components

In AEM, components:

- Are the structural elements that constitute the content of the pages being authored.
- Are resource types.
- Contain a collection of scripts that implement a specific functionality to present the content on the website.
- Are modular and reusable units.
- Are developed as self-contained units within one folder of the repository.
- Have no hidden configuration files.
- Can contain other components.
- Work anywhere within any AEM system.
- Have a standardized user interface.
- Have configurable edit behaviour.
- Use dialogs that are built using sub-elements such as Granite UI components for the touch UI and widgets.
- Can be developed using HTML Template Language (HTL) (recommended) or Java Server Pages (JSP).

Core Components

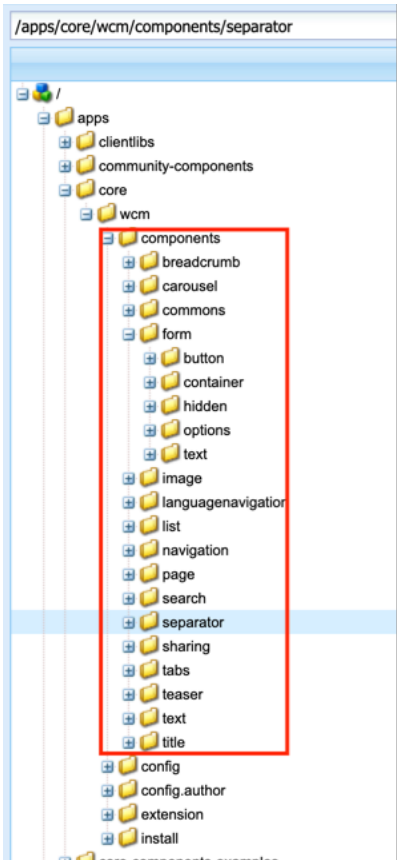
In AEM, Core components:

- Are production-ready components that Adobe provides as a base for site creation
- Provide robust and extensible base components built on the latest technology and best practices adhering to accessibility guidelines
- Are compliant with the Web Content Accessibility Guidelines (WCAG) 2.0 AA standard

You can find that the code on Core component development exists outside of the AEM code base. This enables you to update and enhance the components more often than the product releases. Developers can extend core components to offer custom functionality.

Core Components Library

In AEM, Core components are stored under the /apps section and have components highlighted in the below image.



Note: Visit the GitHub project for Core components to see the latest list of released Core components.

Features of Core Components

The Core components are:

- **Pre-configurable:** Helps page authors define how to use the Core components in templates
- **Versatile:** Helps authors create different types of content
- **Easy-to-use:** Helps authors create and manage content efficiently
- **Production-ready:** Delivers high performance, and are tested comprehensively
- **Accessible:** Complies to the WCAG 2.0 standard, provide ARIA labels, and support keyboard navigation
- **Easy to style:** Implements the Style System, and the mark-up follows Block Element Modifier (BEM) Cascading Style Sheets (CSS) naming
- **Search Engine Optimization (SEO) friendly:** Provides semantic HTML output and schema.org microdata annotations
- **Extensible:** Helps extend the functionality to meet the custom needs without starting from the beginning
- **Open Source:** Helps developers contribute improvements on GitHub (Apache License)
- **Versioned:** Helps developers and administrators determine the compatibility of Core components with their AEM version

Note: Core components are not immediately available to authors. The development team must first integrate the core components into the AEM author environment and preconfigure the Core components from the template editor to make them available for authors. That is why need arise to have proxy components.

Proxy Components

Proxy components define the required component name and group to display to page authors, and refers to a Core component as their super type. Core components must not be directly referenced from the content. To avoid direct reference, the Core components are added to a hidden component group (.core-wcm or .core-wcm-form). This prevents displaying Core components in the Template Editor.

Best Practice: Create proxy components from core components and use the proxy components in your project.

After creating proxy components, you will notice that most components are simply a node with properties to make them unique proxy components. Some proxy components have node structures for further customization. The table below describes some of the node structures.

Components	Node Structure
Carousel component	cq:template: Represents the initial setup of the carousel. This node makes it easier for an author to understand how the component works with two example items.
Image component	cq:editConfig: Determines how AEM renders the image that is dragged onto the component.
Tabs component Teaser component	cq:template: Represents the initial setup for tabs. imageDelegate property: Delegates the image rendering to the core image component.
Forms Container component	new: Inserts a layout container so you can add form components to the container.

Note: The examples above are specific to each component in their respective row, but the nodes could be applied to any proxy component depending on the design requirements of the proxy component.

Exercise 1: Create proxy components

In this exercise, you will create proxy components for the Core components to be used in the training website. After completing this exercise, you can test the AEM site by using different components.

1. In CRXDE Lite, navigate to the /apps/training/components/content folder. If the content folder does not exist, create it.
2. Right-click the content folder and click Create > Create Node. The Create Node dialog box opens.
3. Update the following details:
   a. Name: title
   b. Type: cq:Component
4. Click OK. The Title component is created.
5. Click Save All from the actions bar.

Select the title node, and add the following properties:

Name	Type	Value
sling:resourceSuperType	String	core/wcm/components/title/v2/title
jcr:title	String	Title
componentGroup	String	We.Train

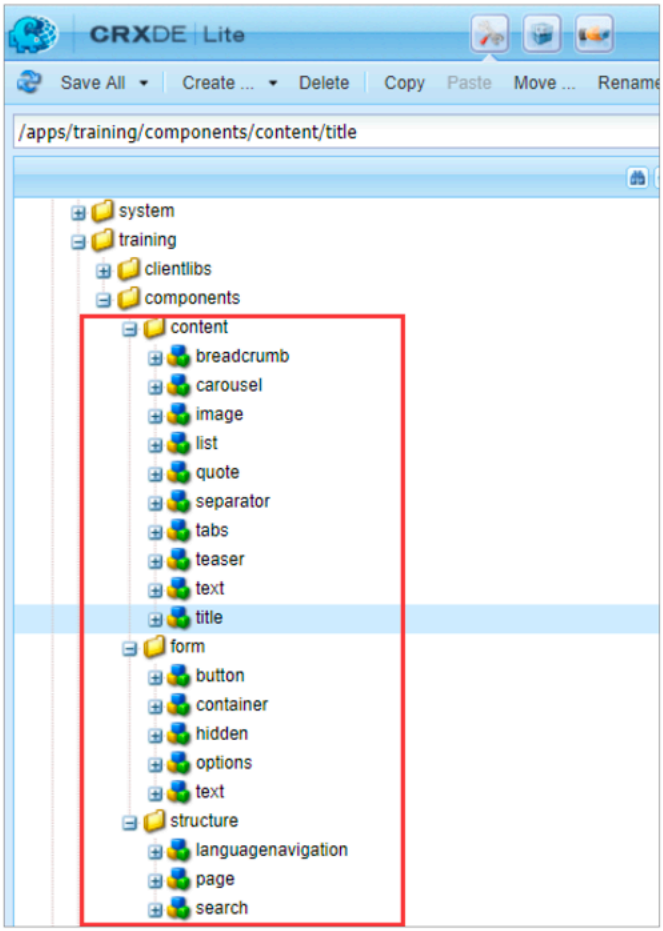
6. Click **Save All** from the actions bar.

Note: In the next step, you will create the rest of the proxy components by installing a content package. Now that you know how to manually create a proxy component, if you would like to manually create these components, repeat steps 2-8 to add the following components:
/**content/breadcrumb***
/content/carousel^*
/content/image^*
/content/list
/content/separator
/content/sharing
/content/tabs^*
/content/teaser
/content/text*
/form/button
/form/container*
/form/hidden
/form/options
/form/text
/structure/search*
/structure/languagenavigation

Note: The Caret (^) symbol next to some of the components indicates that the components have additional node structures that are created with the content package.

Note: The Asterisk (*) next to some of the components indicates that the components have an accompanying client library that must be added. You will add these in a later exercise.

7. To install the We.Train proxy components, click http://localhost:4502/crx/packmgr/index.jsp. The CRXDE Package Manager opens.
8. Click Upload Package from the actions bar. The Upload Package dialog box opens.
9. Click Browse.
10. Navigate to the exercise folder provided to you and double-click the we-train-proxy-components.zip package.
11. Click OK in the Upload Package dialog box. The package is uploaded.
12. In the we-train-proxy-components.zip section, click Install. The Install Package dialog box opens.
13. Ignore the Advanced Settings and click Install. The package is installed on your instance.
14. In CRXDE Lite, view the proxy components under:
    - /apps/training/components/content
    - /apps/training/components/form
    - /apps/training/components/structure

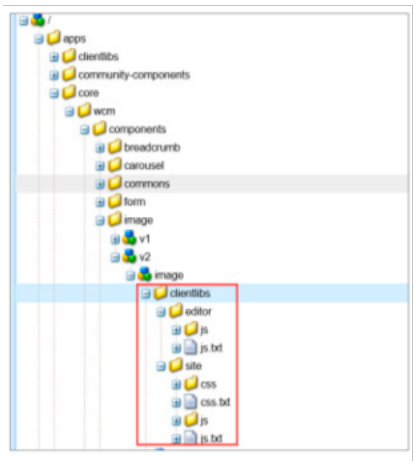


Client Libraries for Core Components

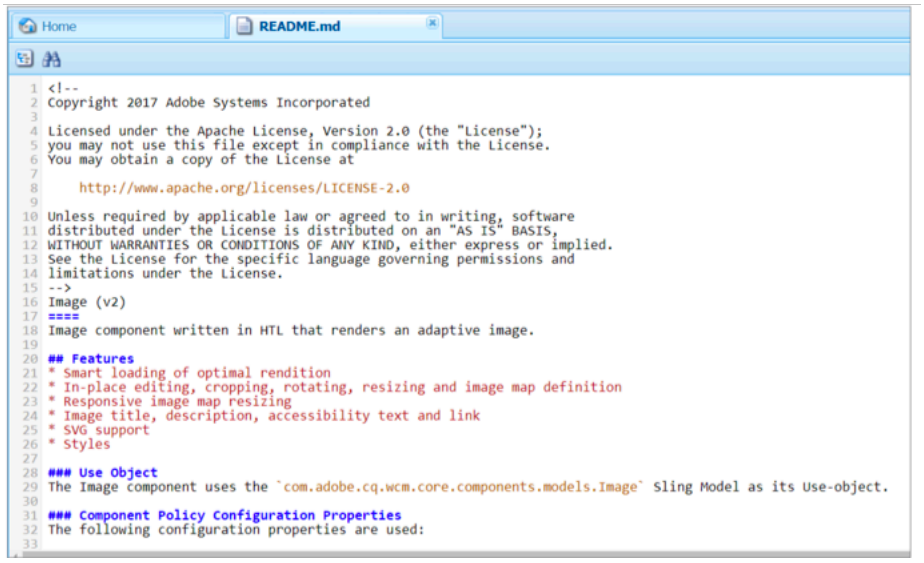
Several Core components have client libraries that must be included on the page for the component to function properly. For example, under **/apps/core/wcm/components/image/v2/image/clientlibs** notice, there are two client libraries—**editor** and **site,** as shown in the below screenshot:



The **editor** clientlib is meant only for the author environment and are typically included by the component dialog. The site clientlib must be included in the project's clientlib for the component to work properly.



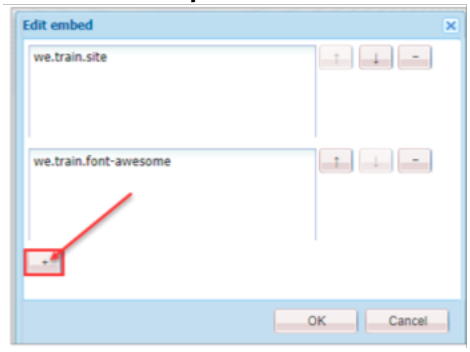
For each Core component, you can view the README.md file to understand what each clientlib does. For example, navigate to **/apps/core/wcm/components/image/v2/image/README.md** to view the details about the Image Core component.



Exercise 2: Add core component client libraries

As you are using multiple core components as proxy components, you must include the required CSS from the core components. You will add these to the clientlib-base client library with the core component category names.

1. In CRXDe Lite, navigate to /apps/training/clientlibs/clientlib-base.
2. Double-click the embed property. You should see the clientlibs you added in a previous task.
3. To add more category names, click the plus (+) icon, as shown, and add the **core.wcm.components.breadcrumb.v2** category name.



4. Repeat step 3 to add the following category names as separate entries:

   core.wcm.components.carousel.v1

   core.wcm.components.image.v2

   core.wcm.components.tabs.v1

   core.wcm.components.search.v1

   core.wcm.components.form.text.v2

   core.wcm.components.form.container.v1

5. Click Save All to save the changes.

Content Policies

The content policies define the design properties of a component, such as the available components or dimensions. These policies are applicable to the template and the pages created with the template. Template authors can define and update the content policies in the template editor when creating the editable templates.
The property cq:policy, on the root node provides a relative reference to the content policy for the page's paragraph system:
/conf/<your-folder>/settings/wcm/templates/<your-template>/policies/jcr:content/root
The property cq:policy, on the component-explicit nodes under root, provides links to the policies for the individual components.
The actual policy definitions are stored under:
/conf/<your-folder>/settings/wcm/policies/wcm/foundation/components

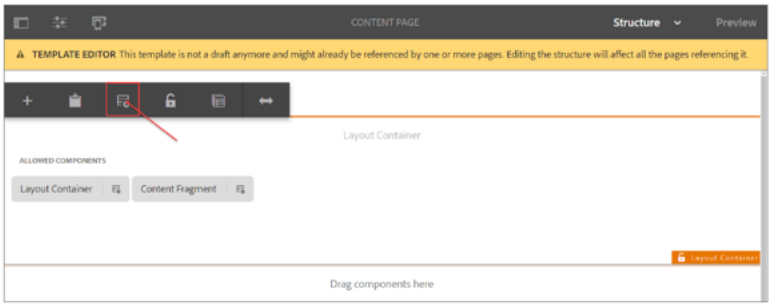
Note: Storing policies under a general location helps the reusability of content policies across different templates.

A common use case for content policies is when you want to use the same components across all templates. Content policies help set up allowed components for the Layout Container component on one template and then reuse that policy for all other templates that should use the same component set. If the policy ever needs to be updated with more components, you can modify the allowed components once and all templates will reflect the change.

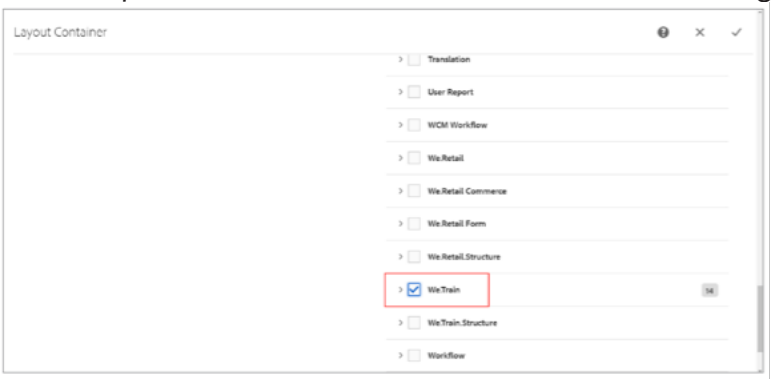
Exercise 3: Add proxy components to the template

In this exercise, you will add the proxy components to the development template to help authors add components to a page. The page authors cannot edit the Layout Container [root] on the page. You will add a Layout Container to the template and specify the components that an author can use in the page through the Layout Container's content policy.
To add proxy components to the template:

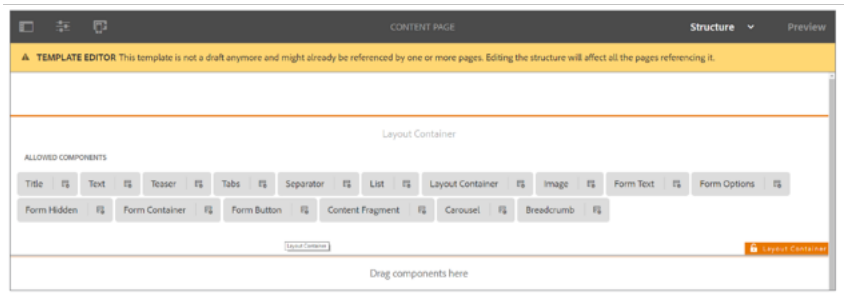
1. Click http://localhost:4502/aem/start.html.
2. Click **Adobe Experience Manager** > **Tools** > **Templates**. The **Templates** console opens.
3. Click the **We.Train** folder, and select the **Content Page** template by clicking the **Select** icon (checkmark). The template is selected.
4. Click **Edit (*e*)** from the actions bar. The template opens in a browser tab.
5. Select the inner **Layout Container** and click the **Policy** icon from the component toolbar, as shown. The **Layout Container** wizard opens.



6. In the **Policy** section, ensure **We.Train Content Policy** is selected. You created this policy when you created the template earlier.
7. In the Properties section, scroll down and select the We.Train group checkbox, as shown:



8. Click Done (checkmark icon in the upper right). This will add the newly created proxy components to the Layout Container from the We.Train group. Any new components with the componentGroup=We.Train property, will be added to the policy automatically.



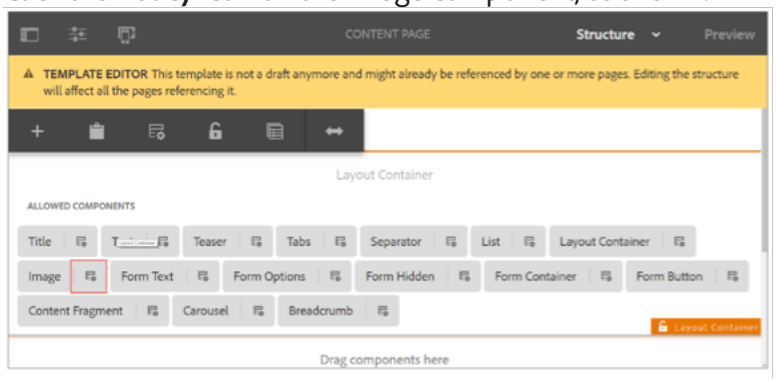
Exercise 4: Add content policies to proxy components

Now that you have proxy components for the Core components, you can make use of the built-in features of the proxy components. In this exercise, you will set up a content policy for the image component that will restrict uploading images from an author's local folder. Authors can only use images from the DAM.

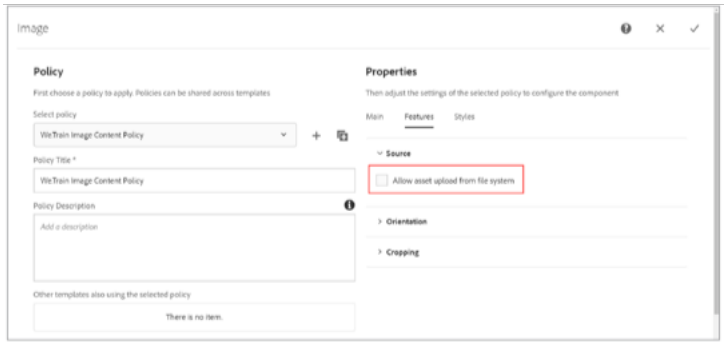
1. Navigate to **Tools** > **Templates** > **We.Train** > **Content Page** in the AEM author instance.
2. In the inner Layout Container (not the root Layout Container), notice that the proxy image component was automatically added. This is because the componentGroup is We.Train. Notice that

there is a policy icon next to the component name. This means that the component has a configurable policy.

3. Click the **Policy** icon on the Image component, as shown:



4. In the Policy section, type **We.Train Image Content Policy** in the **Policy Title** field.
5. In the **Properties** section, ensure you are on the **Main** tab.
6. Under **Widths**, click Add, and enter **1024.**
7. In the **JPEG Quality** field, enter **100.**
8. On the **Features** tab, in the **Properties** section, ensure the **Allow asset upload from file system** checkbox is not selected, as shown:



9. Click **Done** (checkmark) to save changes. Later, when you test the image component, notice how you can only drag and drop images from the DAM onto the image component and cannot upload images from your local file system.

Developing Core Components

Core components provide robust and extensible base components, which are used by developers to implement the business requirements.

As a developer, before you start working with Core components:

1. View the AEM GEMS session to understand the features of Core components and how to leverage them in AEM

2. Check the component library to view the current release of Core components and different examples on how to use the Core components in your project
3. Follow Adobe's recommendations on when and how to use Core components in your new or existing projects
4. Start developing AEM Sites with Core components through the WKND tutorial that demonstrates how to implement Core components in AEM

Note: The link to the AEM GEMS session is: https://helpx.adobe.com/experience-manager/kt/eseminars/gems/AEM-Core-Components.html

Note: The link to the WKND tutorial is: https://helpx.adobe.com/experience-manager/6-4/sites/developing/using/getting-started.html

Note: To keep up to date on the latest changes to the Core components, watch/follow the Core components repository on GitHub: https://github.com/adobe/aem-core-wcm-components

Author with Core Components

Core components provide the rich-authoring functionality and several advantages to authors when adding content to pages. Components are available on the Components tab of the side panel of the page editor when editing a page.

Components are grouped according to categories called component groups to easily organize and filter the components. The component group name is displayed along with the component in the component browser, and it is also possible to filter components by group to easily find the right component.

Pre configuring Core Components

With Core components, a template author can configure a number of capabilities through the Template Editor or in the design mode. For example, if an image component should not allow image upload from the file system or if a text component should only allow certain paragraph formatting features can be enabled or disabled with a simple click.

As the Core components can be pre-configured by template authors to define what options are allowed as part of a template, and further configured by the page author to define actual page content, each component can have options in two different dialogs:

- Edit dialog: Provides the options that a page author can modify during page editing for the placed components. For example, formatting of content text and rotating an image on a page.
- Design dialog: Provides the options that a template author can modify when configuring a page template. For example, text formatting and image in-place editing options.

The styles of most Core components can be defined by using the AEM style system.

- A template author can define which styles are available for a particular component in the design dialog of that component.

- The content author can then choose which styles to apply when adding the component and creating content.

Note: As a developer, understanding the Core components authoring capabilities and leveraging the Core components is vital for rapid project development. After you know the authoring features of the Core components, you can create a development plan for designing and customizing the components further, if required. To test the authoring capabilities of the Core components, you must create proxy components and add the Core component client libraries to your project to test the authoring capabilities.

Exercise 5: Add core components

Core components provide the toolset to quickly start your projects with minimal component development. In this exercise, you will explore the different Core components you added to your project. While working with these Core components, you will notice that they do not have a design yet. By using client libraries, you can easily apply designs to the core components—as they follow the BEM notation. This is a good exercise to come back to after completing this class to fully understand the capabilities of some of the core components.

This exercise includes the following tasks:

1. Create a Page
2. Add Core components

Task 1: Create a page

1. In your AEM author instance, navigate to the **Sites** > **We.Train** > **English** page.
2. Click **Create** > **Page**. The **Create Page Template** wizard opens.
3. Select the **Content Page** template and click **Next**. The **Create Page Properties** wizard opens.
4. In the **Title** field, type **Core Components**, and click **Create**. The **Success** dialog box is displayed.
5. Click **Open** to open the page.

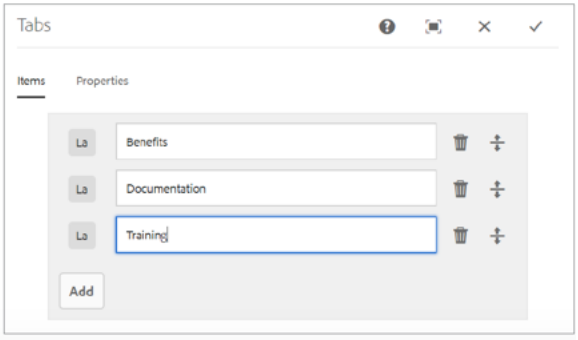
Task 2: Add Core components

1. On the page you created in the previous task, ensure you are in the **Edit** mode, and click the **Toggle Side Panel**.
2. Click the **Components** icon in the left panel. The list of available components is displayed.
3. Drag the **Title** component onto the **Drag components here** area.
4. Double-click the **Title** component you added to edit it. The **Title dialog** box opens.

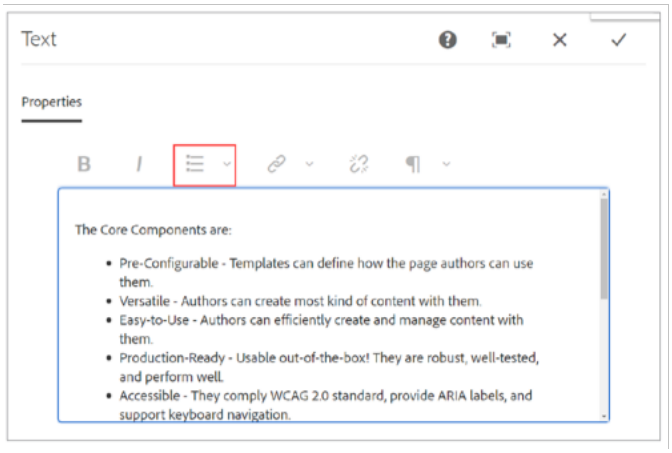
5. In the **Title** field, type **Learn about Core Components**, and click the **Done** (checkmark) icon in the upper right. The title is updated.
6. From the Components browser in the left panel, drag the Tabs component onto the Drag components here area. Two tabs, Tab 1 and Tab 2, are added, as shown:



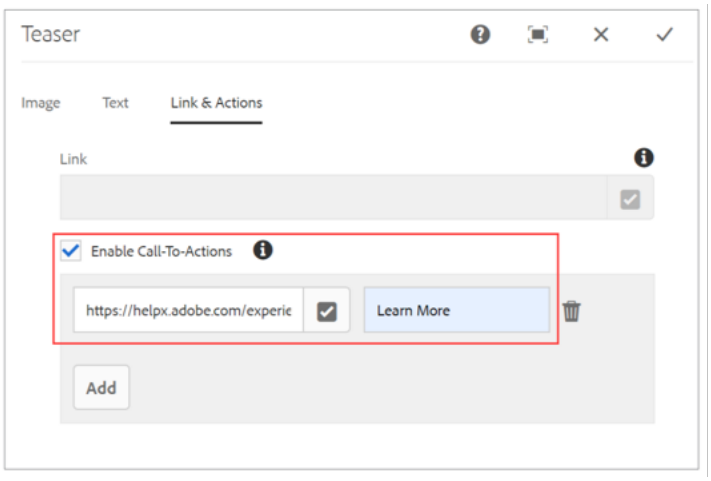
7. Double-click the tabs component. The Tabs dialog box opens.
8. On the default Items tab, rename Tab 1 as Benefits.
9. Rename Tab 2 as Documentation.
10. Click Add. The Insert New Component dialog box opens.
11. Click Layout Container. A new tab is added.
12. Name the new tab, Training. The Tabs dialog box should look, as shown:



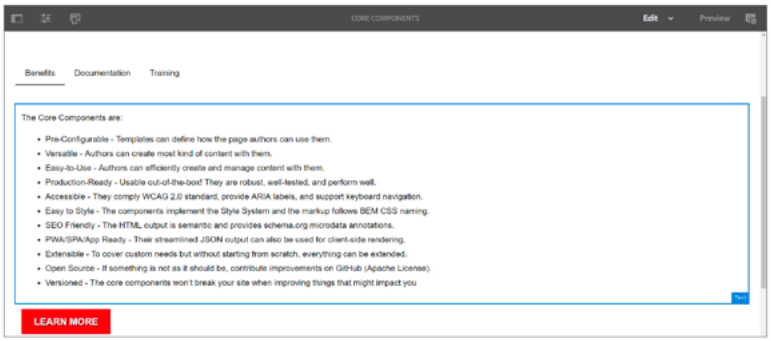
13. Click Done (checkmark in the upper right) to save the changes. The three tabs are added to the page.
14. On the Benefits tab, click the Drag components here area.
15. Click the Insert component icon (the + icon) and add a Text component.
16. Navigate to exercise file provided to you and copy the content from Benefits.txt.
17. On the AEM Components page, double-click the Text component. The Text dialog box opens.
18. Paste the content and apply the bullet list format to the text, as shown:



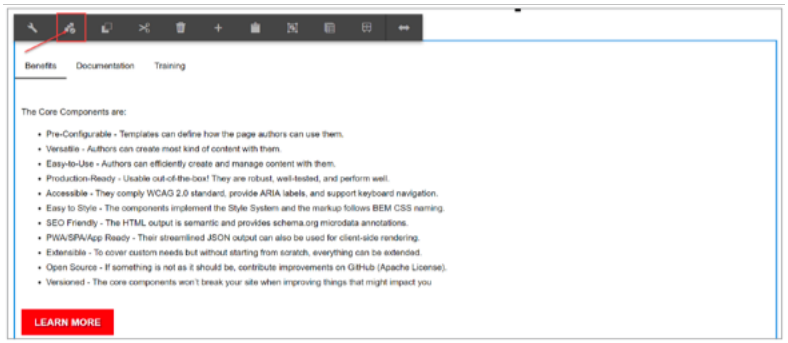
19. Click Done to save the changes. The text is added to the Benefits tab.
20. Click the Drag components here area below the text you added in the previous step.
21. Click the Insert component icon (the + icon) and add a Teaser component.
22. Click the Teaser component and click the Configure icon (the wrench icon). The Teaser dialog box opens.
23. On the Link & Actions tab, select the Enable Call-to-Actions checkbox.
24. In the Link field, type https://helpx.adobe.com/experience-manager/core-components/using/introduction.html.
25. In the Text field, type Learn More. The Teaser dialog should look, as shown:



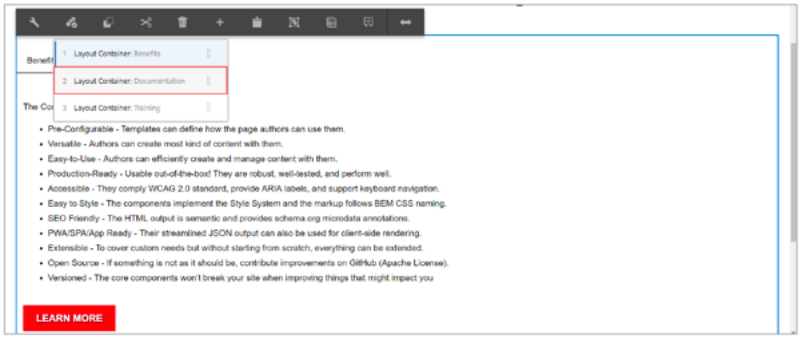
26. Click Done (checkmark icon) to save the changes. The Learn More button is added to the page. The Benefits tab should look, as shown:



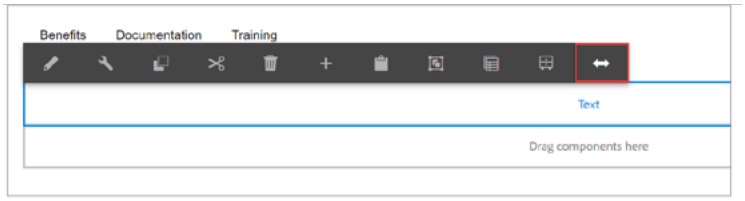
27. To add components and content on the Documentation tab, click the Tabs component and click Select Panel, as shown:



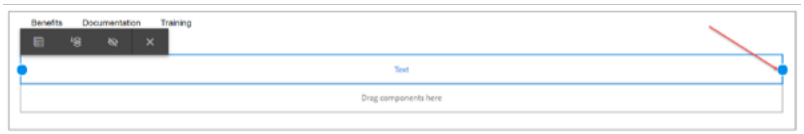
28. Click Layout Container: Documentation, as shown. The components and content you insert now will be added under Documentation.



29. On the Documentation tab, click the Drag components here area.
30. Click the Insert component icon (the + icon), and add a Text component.
31. Click the Text component, and click Layout, as shown. The layout is now editable.



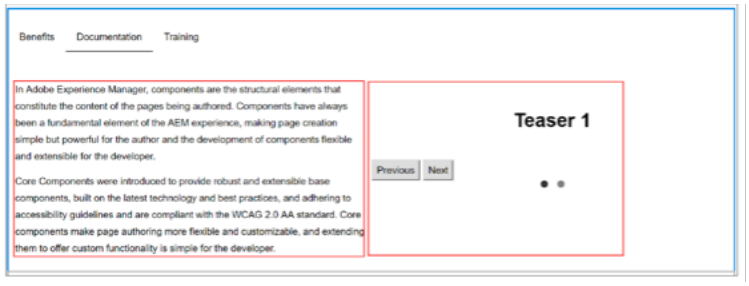
32. Drag the blue circle and adjust the layout to 6/12 columns, as shown:



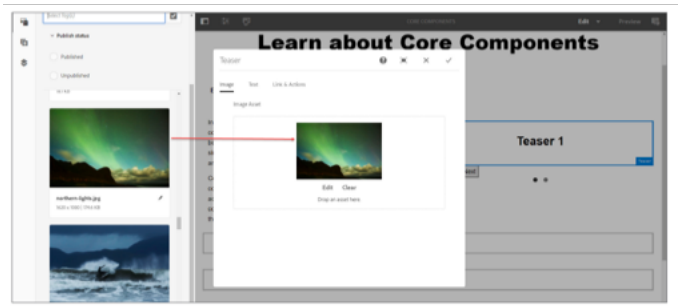
33. Click Close. The component should look, as shown:



34. Navigate to the exercise file provided to you and copy the content from Documentation.txt.
35. Click the Text component in AEM, and click the Edit icon (pencil icon) to add content.
36. Paste the content you copied in step 34 and click Done. The text is added under the Documentation tab.
37. Add a Carousel component under the Text component in the Documentation tab.
38. Click the Carousel component, and click Layout. The layout is now editable.
39. Drag the blue circle and adjust the layout to 6/12 columns. The two components are now placed next to each other, as shown:

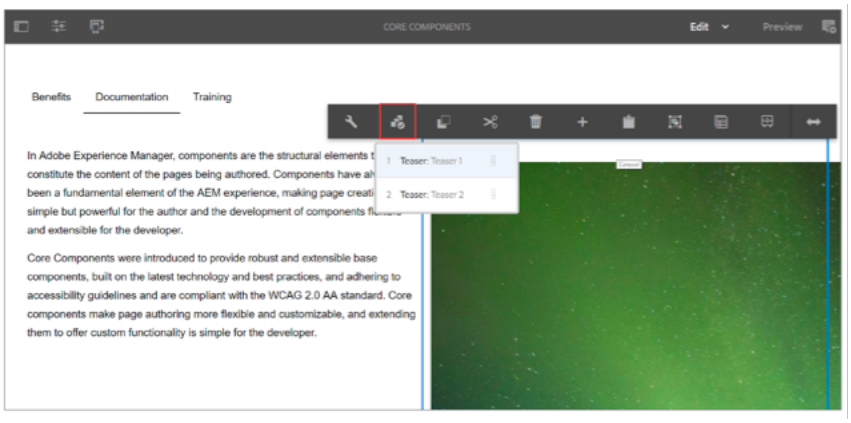


40. Select the Teaser 1 dialog box and click Configure. The Teaser dialog box opens.
41. On the Image Tab, drag an image from the Assets tab onto the Drop an asset here area. The image is added.

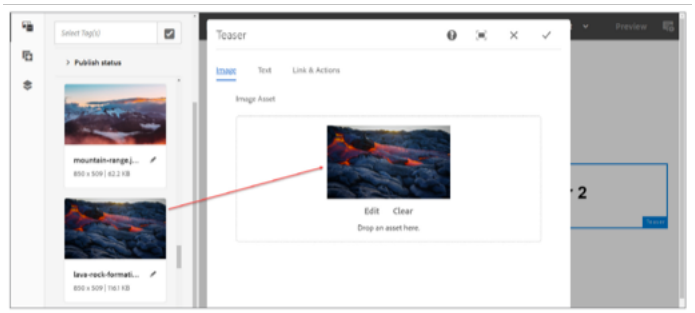


42. On the Text Tab, select the Get title from linked page checkbox.
43. On the Links & Actions tab, select the Enable Call-to-Actions checkbox.

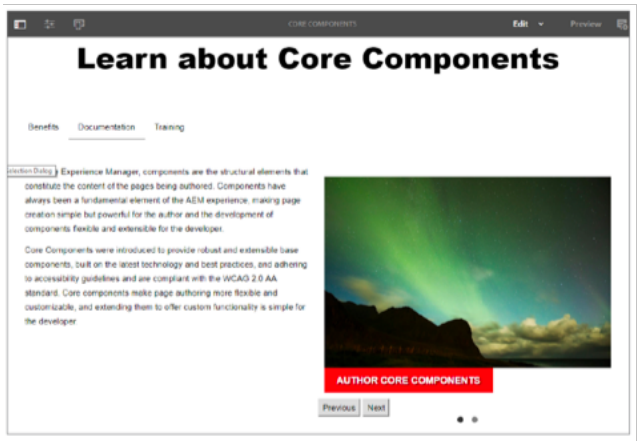
44. In the Link field, enter https://helpx.adobe.com/experience-manager/core-components/using/authoring.html
45. In the Text field, enter Author Core Components.
46. Click Done.
47. Select the Carousel component and click the Select Panel icon, as shown:



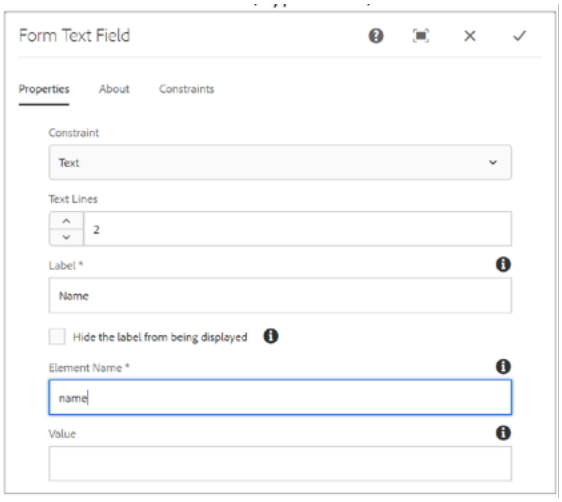
48. Click Teaser: Teaser 2 to add content under Teaser 2.
49. Click the Toggle Side Panel icon to access the options in the left panel.
50. Select Teaser 2 dialog and click the Configure icon. The Teaser dialog box opens.
51. From the Assets tab, drag an image onto the Drop an asset here area.



52. On the Text tab, select the Get title from linked page checkbox.
53. On the Links & Actions tab, select the Enable Call-to-Actions checkbox.
54. Click the Done icon (checkmark in the upper right).
55. In the Link field, type https://helpx.adobe.com/experience-manager/core-components/using/developing.html
56. In the Text field, type Develop Core Components.
57. Click Done to save the changes. The Documentation tab should look, as shown:

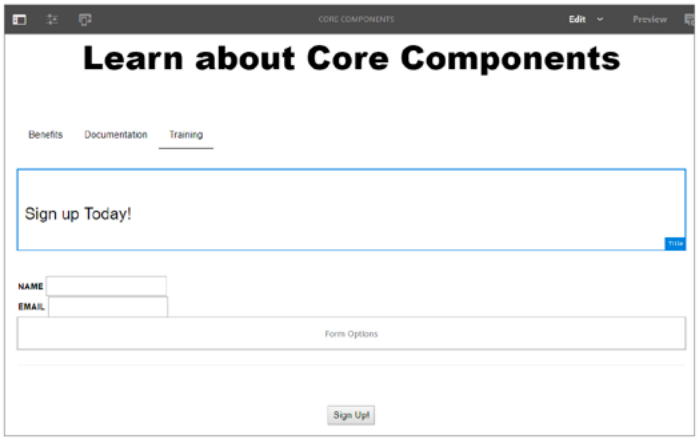


58. Select the Tabs component, click the Select Panel icon, and click Layout Container: Training to add components and content under the Training tab.
59. Click the Drag components here area and add the Title component.
60. Double-click the Title component you added now to edit it. The Title dialog box opens.
61. In the Title field, type Sign up Today!
62. From the Type/Size drop-down menu, select H3.
63. Click Done to save the changes.
64. Add a Form Container component, select it, and click Configure. The Form Container dialog box opens. Notice under Content Path that the submitted form is saving to /content/usergenerated/*. This could be an external database as well.
65. Click Close.
66. Add a Form Text component, select it, and click Configure. The Form Text Field dialog box opens.
67. In the Label field, type Name.
68. In the Element Name field, type name, as shown:



69. Click Done to save the changes.

70. Add another Form Text component to the page, select it, and click Configure. The Form Text Field dialog box opens.
71. In the Label field, enter Email.
72. In the Element Name field, enter email.
73. Click Done to save the changes.
74. Add a Form Options component to the page, select it, and click Configure. The Form Options Field dialog box opens.
75. in the Title field, enter Audience.
76. In the Name field, enter audience.
77. Click Add under Option Entries and enter Author in the Text field.
78. Click Add and enter Developer in the Text field.
79. Click Done to save the changes.
80. Add a Separator component to the page. A line that separtes the two components are added.
81. Add a Form Button component below the Separator component, select it, and click Configure. The Form Button dialog box opens.
82. In the Title field, enter Sign Up!
83. Click Done to save the changes. The Training tab should look, as shown:



84. To view your completed work, click Preview in the upper-right corner of the page. Click all tabs and links to check if all the elements are working correctly.